



DGUI-HyperMem: A Reasoning-Augmented Hybrid Memory Service for LLM Agents on Serverless Infrastructure

Wan Mohd Azizi bin Wan Hosen

CTECX Development & Research 

dgui-hypermem.deckergui.my

ABSTRACT

An agent that forgets is an agent that pays twice. This paper describes DGUI-HyperMem (DeckerGUI HyperMemory), a long-term memory service for LLM agents that runs entirely on a serverless edge runtime, is addressed over the Model Context Protocol, and is designed so that an agent’s memory is queried *by the agent itself* rather than replayed as prompt text. The system fuses approximate-nearest-neighbour search over a 768-dimensional vector index with BM25 over an external-content FTS5 index by reciprocal rank fusion, then submits the fused shortlist to a reasoning layer that re-ranks and re-scores it. A write path assigns every memory a type, a salience and a durability judgement, suppresses low-signal content behind a calibrated salience gate, and automatically supersedes memories that a newer write contradicts. We contribute a description of the architecture and its exact scoring and gating constants; a privacy argument for why redaction must happen at enqueue rather than at export, motivated by an incident in which a passkey reached a public dataset; and a simulated retrieval evaluation built on the production embedding model. That evaluation returns a negative result about the deployed configuration: equal-weight rank fusion is *worse* than the dense channel alone (0.378 against 0.418 nDCG@10), because reciprocal rank fusion discards score magnitudes and so lets a weak lexical channel dilute a strong dense one. Re-weighting the lexical channel to 0.3 recovers the loss and edges past the best single channel (0.422, with recall@10 rising from 0.598 to 0.624), which localises the defect to the weights rather than to the fusion. We also find that the judgement stage is not a precision win but a recall and difficulty trade, and we report it as such. The evaluation is explicitly a simulation of long-horizon engineering-agent memory and is not a DeepSWE result; we state the distinction and the threats to validity it carries. The reasoning corpus exported by the service is public, which makes the curation pipeline itself reproducible.

Keywords agent memory · reciprocal rank fusion · Model Context Protocol · serverless retrieval · knowledge curation

1 Introduction

Retrieval-augmented generation established that grounding a model in external text beats memorising it. Agent memory asks the same question about the agent itself. Across a long session, and across sessions, an agent accumulates facts, decisions, stated preferences, and hard-won diagnoses; on the next task those observations are usually not in the context window, and if they are not retrieved they are simply lost. The cost is not only accuracy. Re-deriving a decision the agent already made wastes tool calls, and re-discovering a diagnosed failure is the most expensive kind of mistake an autonomous agent makes, because the symptom has usually already been paid for once.

The mechanical solution is a vector store with a similarity search. It is a good solution to the wrong question. Real engineering memory is not uniformly semantic. A developer asking “where is the migration runner” wants

a path match, and no embedding model is reliably better at that than a token match. A developer asking “why did we pick Kafka here” wants a rationale, and the rationale rarely shares content words with the question. A single dense index handles the second case badly and the first adequately; a single lexical index does the reverse. The interesting engineering is in what happens between the two channels and after them.

DGUI-HyperMem is a deployed memory service that takes this seriously. It is a single Cloudflare Worker binding SQLite (D1), a vector index (Vectorize), and a model runtime (Workers AI), exposed to any MCP-capable client[Anthropic, 2025] at one URL. Its central claim is that a memory service should not merely *retrieve* but should *judge*: a reasoning layer inspects the fused shortlist and decides which candidate actually answers the question. Every decision that reasoning layer makes is recorded,

redacted, and exported as training data, so the judgement improves rather than being rewritten each deployment.

1.1 Contributions

1. An architecture and an exact transcription of the deployed retrieval and curation constants — reciprocal rank fusion at $K = 60$, the salience gate at 3.0, the supersede threshold at 0.80, and the two linear scoring blends — so that the system is reproducible rather than merely described (sections 4 and 5).
2. A privacy argument for redaction at enqueue. Redacting at export is too late, and we show why with an incident in which a real credential was written to a public dataset by a pipeline whose only redaction pass ran afterwards (section 7).
3. A simulated retrieval evaluation over a synthetic long-horizon workload, using the same embedding model as the deployed index, that varies the fusion weights and the re-ranking stage independently and reports where each one loses accuracy rather than where it gains it (section 8).
4. A positioning against other agent-memory systems along axes that matter operationally rather than aspirationally (section 2.2).

We make no claim to beat a purpose-built memory service on a shared benchmark. We make a narrower claim, which is that the design decisions described here are coherent, that their constants are calibrated against observed behaviour, and that the component that matters most — a judgement stage over the fused shortlist — is the one most often omitted from comparable systems.

2 Background

2.1 Two retrieval channels and their failure modes

Let D be a memory store, q a query, and d a memory. A lexical ranker scores

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \frac{tf(t, d) (k_1 + 1)}{tf(t, d) + k_1 \left(1 - b + b \frac{|d|}{|q|}\right)}, \quad (1)$$

with $k_1 = 1.2$ and $b = 0.75$. Its strength is exact-match on rare tokens: file paths, identifiers, error strings. Its weakness is that it needs the query and the memory to share a token, so a paraphrase is invisible to it. A dense ranker scores $q^\top d / (\|q\| \|d\|)$ over an embedding, and behaves in the mirror-image way. Our evaluation quantifies the mirror image directly: over paraphrase queries whose gold memory shares no content word with the question, lexical retrieval scores 0.164 nDCG@10 while dense retrieval scores 0.713 (section 8). That gap is the entire motivation for running two channels, and section 8 also shows that naively fusing them destroys most of it.

Reciprocal rank fusion [Cormack et al., 2009] combines ranked lists without requiring comparable scores, which is

exactly the situation here: BM25 magnitudes are corpus-dependent and cosine magnitudes are model-dependent, so any linear blend of the two raw scores would need per-corpus calibration. RRF instead consumes rank positions:

$$\text{RRF}(d) = \sum_{\ell \in \mathcal{L}} \frac{1}{K + \text{rank}_\ell(d)}, \quad (2)$$

over rank lists $\mathcal{L}$, with $K = 60$. The constant damps the head of each list, which prevents a single channel’s top hit from dominating purely by being first.

2.2 Agent memory systems

Several mature systems store memory for agents. We compare them along the axes that determine whether a service can be run and trusted, and defer the full table to table 4.

mem0 offers an extract-and-store pipeline over a vector store with optional BM25 and entity boosting, and reports strong long-conversation benchmarks [Chhikara et al., 2025]. Its April 2026 algorithm revision is explicitly single-pass: memories accumulate and nothing is overwritten, which is a deliberate choice against the cost of an agentic loop at write time. Its read path fuses several signals but does not spend a model call on re-ranking.

Zep [Zep Software, 2025] maintains a temporal knowledge graph and exposes it through a managed cloud service, with Graphiti as the open-source engine. Temporal structure is a real advantage for questions about state and ordering; it also imposes a graph-extraction step and an entity-resolution problem that a row-and-vector store does not have.

Letta [Letta AI, 2025] treats memory as an agent feature rather than a service: the agent is the stateful object, and memory blocks are part of its context construction. This is a different integration contract from a memory endpoint another tool can call.

Cognee and the reference *MCP knowledge-graph memory server* [Model Context Protocol, 2025] bracket the design space at the other end. The latter is deliberately minimal — entities, relations, observations, held in a local JSON file — which makes it an excellent teaching artifact and a poor production service.

The gap we occupy is specific: a memory service that is (i) reachable over MCP, (ii) self-hostable on infrastructure the operator already pays for, (iii) hybrid rather than purely dense, (iv) willing to spend a reasoning call to reorder a shortlist, and (v) transparent about the training data its own reasoning generates.

3 System Overview

Figure 1 shows the deployed topology. A Cloudflare Worker is the entire service: there is no container, no orchestrator, and no separate inference service. Three bindings carry the state.

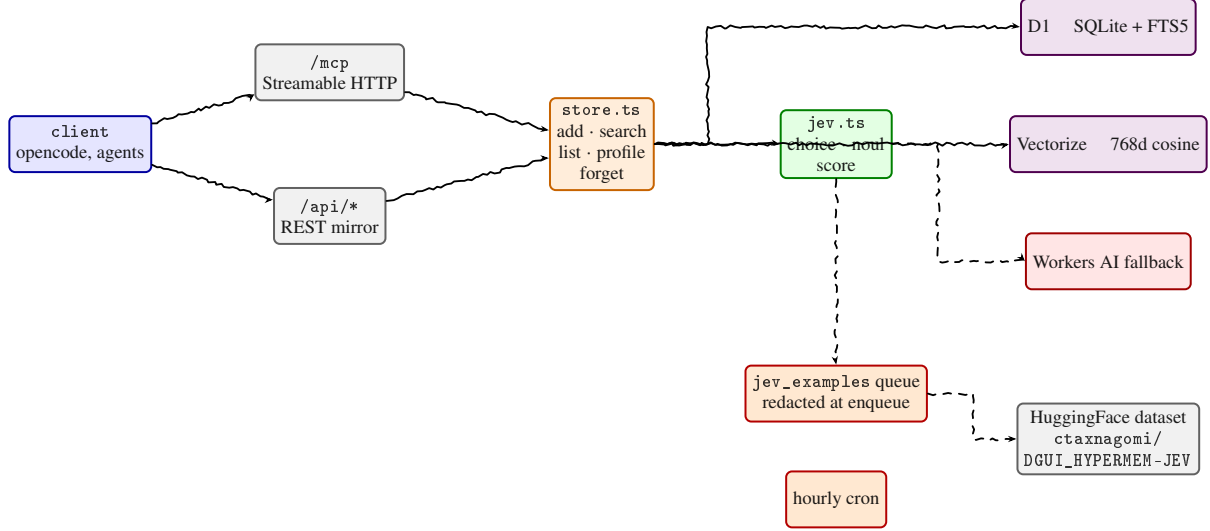


Figure 1: Deployed topology. Solid arrows are request paths; dashed arrows are write-side analysis and asynchronous dataset export. The two protocols are equivalent surfaces over one implementation, so a non-MCP client loses nothing but the tool schema.

D1 (SQLite). Rows of memory, plus an external-content FTS5 table kept in sync by insert, update, and delete triggers, so the lexical index cannot drift from its source of truth. porter unicode61 stemming makes the lexical channel insensitive to inflection. [Cloudflare, 2025a]

Vectorize. 768-dimensional vectors, cosine metric, filtered by scope and status at query time so that superseded rows leave both channels at once. [Cloudflare, 2025b]

Workers AI. The reasoning backend, used both as the primary judge and as the fallback when the primary is unavailable. [Cloudflare, 2025c]

The service exposes nine tools (table 1). The set is deliberately small because an agent’s tool budget is finite and a memory service competes with the agent’s actual work for that budget.

4 Write Path

Figure 2 is the write path. Four things happen, in this order, and the order is the design.

Refusal before analysis. Content matching a prohibited pattern — live Stripe or GitHub credentials, AWS keys, PEM private keys, exploit references, or child sexual abuse material — is rejected before any model is called and before anything is stored. This is a hard refusal, not a redaction: the content is never written, so it cannot reach the dataset pipeline.

One reasoning pass, three judgements. Section 6 describes the layer. For the write path it yields a type, a durability score, and a salience score.

Table 1: The MCP tool surface. Destructive tools are annotated as such so an agent can gate on it; `sync_jev_dataset` is the only tool with an effect outside the caller’s own scope.

Tool	Purpose
add	Store a memory; assigns type and salience, supersedes contradicted rows
search	Hybrid recall then re-rank; returns scored memories
suggest	Detect repeats, find related rows, propose generation
list	Browse a scope, newest first
profile	Totals, counts by type, top tags, mean salience
forget	Delete by id, by query, or a whole scope (destructive)
help	Describe the server and its tools
sync_jev_dataset	Flush queued decision rows to the public dataset
jev_queue_stats	Pending, uploaded, and failed counts by use case

The salience gate. Salience is an ordinal score on a five-point scale, so the natural question is where to cut. The deployed default is 3.0, and the comment in the source is worth repeating because it records a calibration failure: a threshold of 4.0 “filtered everything in this dataset”, while 3.0 sits at the model’s median. A gate set from intuition rather than from the observed score distribution is the single most common way a curation layer becomes a memory *deleter*, because it does not fail loudly — it simply never stores anything.

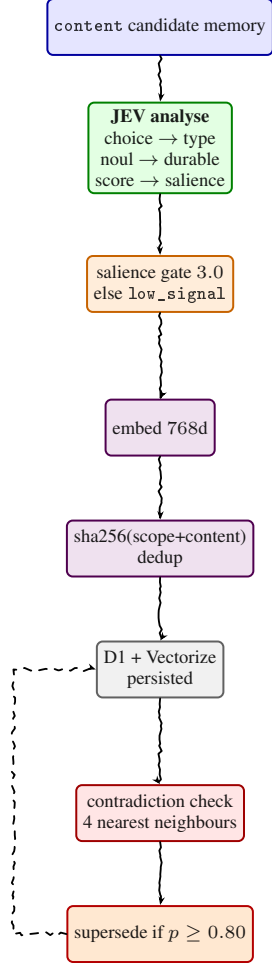


Figure 2: Write path. The reasoning call precedes persistence, so a memory is never stored without its judgements; the contradiction check follows persistence, so it can compare against what is already there.

Deduplication by content hash. The hash is taken over scope and normalised content, with a unique index on the pair. A repeated add updates the existing row rather than creating a second one.

Contradiction and supersession. After persisting, the system queries the four nearest neighbours and asks the reasoning layer for the probability that the incoming memory contradicts each. Above $p \geq 0.80$ the earlier row is marked superseded, its vector is deleted, and the supersession is logged. Rows are never destroyed, so a superseded memory is still recallable by id, which is what makes section 8’s temporal queries answerable at all.

5 Read Path

Figure 3 is the read path: two channels, a fusion, a judgement, and a blend.

Two channels, filtered identically. Both channels are scoped and filtered to `status = 'active'`, so a superseded row is invisible to both. Filtering only one channel would let a retired memory re-enter through the other, which is precisely the staleness failure the supersede logic exists to prevent.

The lexical channel builds an FTSS MATCH query from up to sixteen stemmed tokens joined by OR. The dense channel issues a filtered vector query with the same limit.

Fusion. Equation (2) with $K = 60$ over the two rank lists. The fused score is normalised by the maximum fused score in the candidate set, giving $F \in [0, 1]$, and the top $\max(2k, 12)$ candidates form the shortlist.

Judgement. The reasoning layer scores every shortlisted candidate for relevance on the same bounded scale the write path uses. This is the one step that costs a model call, and it is bounded by the shortlist rather than by the store.

The blend. Let $S = \text{clamp}((\text{salience or } 2)/4, 0, 1)$ be the normalised salience and J the bounded relevance judgement. The final score is

$$\text{score}(d) = \begin{cases} 0.85 F(d) + 0.15 S(d), & \text{reasoning unavailable,} \\ 0.50 F(d) + 0.40 J(d) + 0.10 S(d), & \text{otherwise.} \end{cases} \quad (3)$$

The weighting is a considered trade rather than a tuned constant. Fusion is the component with the most evidence behind it, so it carries half the weight even when a judgement is available; salience is retained at a small weight even when fusion is the only signal, because a memory the model considered important is weak evidence that a user who asked about it wanted it. Every term is in $[0, 1]$: this is a design constraint on the reasoning backend, not an afterthought, and section 8 shows that feeding an unbounded score into the 0.40 slot distorts the blend.

Results are sorted by eq. (3) and truncated to k . Returned rows have their access counters touched and the search is logged with candidate and result counts, which is what makes the profile tool and the queue statistics possible without a second telemetry path.

6 The JEV Reasoning Layer

The layer exposes three primitives and nothing else fig. 4. The restriction is deliberate: a small, fixed verb set makes the recorded training rows comparable to live inputs, because the instruction block is the same object in both.

Seven types. Fact, preference, decision, project, solution, architecture, and conversation. The *conversation* type exists so that session-only chatter has somewhere to go; a memory store with no such category either keeps trivia forever or throws it away, and both failure modes are worse than labelling it.

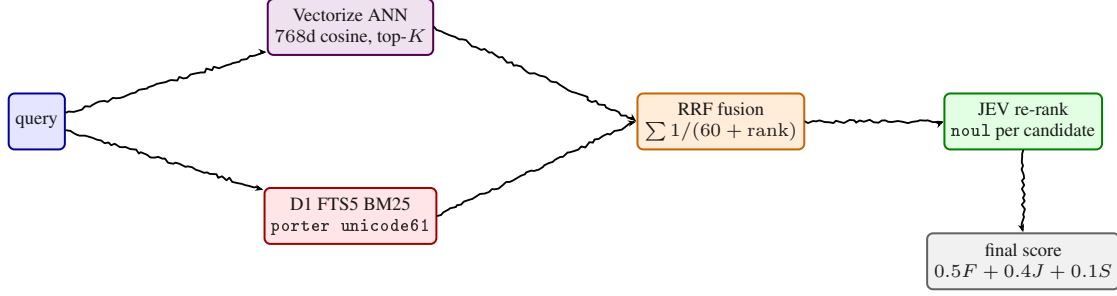


Figure 3: Hybrid recall. The shortlist handed to the reasoning layer is bounded ($\max(2k, 12)$ candidates at the deployed settings), so the cost of judging is capped independently of store size.

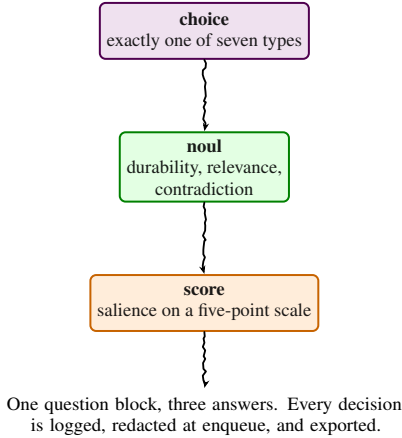


Figure 4: The reasoning primitives. *choice* runs once per memory; *noul* runs per write, per shortlisted candidate, and per contradiction test; *score* runs per write.

Three backends. The layer resolves to a hosted reasoning backend, a Workers AI emulation of the same three primitives, or off. The *off* path is a real mode, not an error: it makes eq. (3)’s first branch the normal operation, which is why that branch exists and why it is specified rather than left as undefined behaviour.

Bounded outputs. Every primitive returns a value in $[0, 1]$. The scoring blend depends on this contract. It is a genuine coupling, not a formality, and it is the kind of detail that is usually left implicit and then violated by the first backend added.

7 Privacy: Redact at Enqueue

The reasoning corpus this service exports is public, by design: it is the training set for the reasoning layer, and a curation corpus nobody can inspect is not a research contribution. That makes the export path a disclosure surface, and it constrains where redaction must happen.

The failure we are arguing about. An earlier revision of this system redacted *at export*. The queue held rows

verbatim; the sync job read them, redacted, and uploaded. A real administrator passkey was stored as memory during ordinary use, the row was queued, and the redaction pass did not match it. The credential was published. The dataset was corrected, but the value had been public, and a rotating credential does not undo a publish.

The generalisation is the argument: redaction at export is a promise about a code path that runs on someone else’s data, in a batch job, at an hour nobody is watching. Redaction at enqueue is a property of the write path, exercised on every row, in the same request that stores it.

How the enqueue pass works. Two layers. Unambiguous provider formats are matched directly: OpenAI *sk-*, Stripe *sk_live-* and *whsec-*, GitHub *ghp-* and *github_pat-*, HuggingFace *hf-*, Cloudflare *cfat-*, Slack *sox*, AWS *AKIA*, Google *AIza*, and JWTs.

The interesting case is the ambiguous one. A generic *credential-assignment* rule requires a credential keyword (*passkey*, *api_key*, *bearer_token*, ...) followed by a delimiter or a bounded copula chain, then a value. Three details make it usable rather than destructive. The separator permits at most four stacked words, which lets it reach “rotating the passkey to new value x” while preventing the rule from chaining across a sentence. A plausibility gate rejects values that cannot be secrets: anything under six characters, bare digits, and anything on a denylist of ordinary technical vocabulary, so that “wrapped in base64” and “the token authentication” survive intact. And the rule reports *which* rule fired, never the value, so a log line is safe to publish.

Identity fields are whitelisted, not blacklisted. Row metadata needed for slicing the dataset — use case, instruction type, provider, model, timestamps — is passed through untouched, and everything else is walked recursively, because free text appears both as top-level JSON strings and nested inside *state*, *questions*, and *answers*. A blacklist of known-sensitive keys would have missed the nested copies; walking non-identity fields does not.

What this does not do. We state plainly: pattern matching is not entropy analysis. A secret in an unfamiliar for-

mat, or one with no credential keyword near it, will pass. This layer raises the cost of an accident; it does not make disclosure safe. The conclusion is architectural — treat any exported corpus as public, and design the pipeline so that being wrong is survivable — rather than a claim that the rules are complete.

8 Evaluation

8.1 What this evaluation is, and is not

DeepSWE [DeepSWE, 2026] is a real software-engineering agent benchmark. Running it requires paid proprietary endpoints and an air-gapped runner. Neither is available to us, and a fabricated score attributed to it would be worse than no score. This section therefore reports a *simulation* of the memory behaviour such a benchmark exercises: persistent facts, stated rationales, symptom-and-fix records, configuration that evolves across sessions, and contradictions between them. We make no claim about DeepSWE performance and our harness is not derived from it.

8.2 Harness

A seeded generator builds a synthetic engineering repository and a memory store of roughly 870 rows across 95 sessions. Twenty subjects are given supersede chains of length two to four, so that answering “what does this use *now*” requires the whole chain to have been maintained rather than one stale row filtered. The remainder is filler chosen to be realistic distractors: decisions, diagnoses, conventions, and low-value chatter that a good system should rank last.

Every query is planted alongside the memory that answers it, so relevance is never model-judged. Queries fall into four classes: *lexical*, which requires an exact rare token such as a path; *semantic*, which is a paraphrase sharing no content word with its gold memory; *temporal*, which asks for the current state of an evolving subject or for an earlier generation; and *pinpoint*, which asks for a rationale or a diagnosis. Each class is generated to a quota before truncation, so that limiting the query count does not silently drop the smallest class: the reported mix is 37 lexical, 39 pinpoint, 37 semantic and 37 temporal per seed, 150 in total, over three seeds.

8.3 What is faithful and what is substituted

Being explicit here matters more than any single number. The dense channel uses BAAI/bge-base-en-v1.5 [Ding et al., 2022] at 768 dimensions — the same model the deployed index uses — run locally. The lexical channel is Okapi BM25 [Robertson and Zaragoza, 2009] over Porter-stemmed tokens, matching the FTS5 configuration in eq. (1). Fusion uses $K = 60$ and eq. (3) is applied as written.

Two things are simulated. The reasoning call is replaced by a local cross-encoder reranker, whose unbounded relevance logits are squashed with a logistic to occupy the bounded `nou1` slot; that column measures the value of *adding a judgement stage over a fused shortlist*, which is the architectural claim, and it is *not* JEV performance. Write-side type and salience assignment, which are reasoning outputs in production, are generated by deterministic rules.

Table 2 reports eight configurations rather than the four that were run in the deployed service, because the first four made the deployed weights look foolish in a way that needed explaining rather than hiding. The ladder separates three variables that the deployed pipeline conflates: which channels contribute, in what proportion, and whether a judgement stage re-orders the shortlist. The `rrf-w` rows are the only configurations not present in production.

8.4 Results

Table 2 is the headline result, and it is not the one we expected. Table 3 gives the per-class breakdown. Four things follow.

The sanity check holds. A recency baseline scores 0.004 nDCG@10, which is the correct behaviour and evidence that the workload is not solvable by ordering rows by write time. A harness that returned the newest memories first would look strong on any corpus with recency bias and would be measuring nothing.

Equal-weight fusion is worse than one of its own channels. Deployed reciprocal rank fusion scores 0.378 nDCG@10, against 0.418 for the dense channel alone. This is a real regression, not noise: the 0.040 gap is more than three times the larger of the two between-seed standard deviations, and the ordering reproduces on all three seeds independently. The per-class table localises it. Fusion keeps almost all of the dense channel’s paraphrase advantage, taking 0.713 down to 0.538, while gaining almost nothing on the classes where lexical retrieval was already competitive.

The mechanism is a property of RRF itself. Each channel contributes $1/(K + \text{rank})$ and nothing else, so a channel that is nearly blind to the query still contributes a full-strength term for whatever it happens to rank highly. The dense channel’s confident top ranks are therefore partly cancelled by whatever the lexical channel promotes into its own top ranks, and on a paraphrase the lexical channel’s top ranks are noise.

Re-weighting recovers the loss, which confirms the diagnosis. If the problem were fusion itself, no weight would fix it. Setting the lexical weight to 0.3 leaves the dense channel’s ranking dominant while retaining the lexical channel’s genuine hits, and the result is the best row in the table on every metric: 0.422 nDCG@10, 0.489 recall@5 and 0.624 recall@10, against 0.418, 0.473 and 0.598 for

Retrieval configuration	R@5	R@10	MRR@10	nDCG@10	P@1
Recency only (<i>control</i>)	0.004 ± 0.006	0.011 ± 0.008	0.002 ± 0.002	0.004 ± 0.003	0.000 ± 0.000
FTS5 lexical channel alone	0.244 ± 0.018	0.362 ± 0.017	0.191 ± 0.013	0.230 ± 0.013	0.142 ± 0.021
Vectorize dense channel alone	0.473 ± 0.036	0.598 ± 0.028	0.364 ± 0.009	0.418 ± 0.005	0.282 ± 0.031
Dense + cross-encoder rerank	0.453 ± 0.036	0.620 ± 0.024	0.361 ± 0.019	0.422 ± 0.014	0.276 ± 0.039
RRF fusion, equal weights (<i>as deployed</i>)	0.438 ± 0.022	0.587 ± 0.019	0.315 ± 0.014	0.378 ± 0.013	0.227 ± 0.022
RRF equal + rerank (<i>deployed pipeline</i>)	0.427 ± 0.022	0.591 ± 0.019	0.310 ± 0.015	0.375 ± 0.016	0.220 ± 0.014
RRF fusion, lexical weight 0.3	0.489 ± 0.019	0.624 ± 0.013	0.359 ± 0.020	0.422 ± 0.016	0.262 ± 0.030
RRF weighted + rerank	0.478 ± 0.026	0.629 ± 0.014	0.349 ± 0.023	0.415 ± 0.021	0.244 ± 0.027

Table 2: Overall retrieval quality as mean $\pm$ population standard deviation over 3 seeds of 150 queries each, against 864 – 874 stored memories. The “recency” row orders the store by write time and recovers almost nothing; it is the control that makes the rest of the table mean something, since a harness returning the newest rows first would otherwise look competitive on any workload with recency bias. Note that the single dense channel and the re-weighted fusion are tied at the top on nDCG@10, that equal-weight fusion is below both, and that the deployed pipeline is lowest of the fusion family. §8 explains why, and what that implies for the deployed weights.

Retrieval configuration	lexical	pinpoint	paraphrase	temporal
Recency only (<i>control</i>)	0.003 ± 0.004	0.013 ± 0.009	0.000 ± 0.000	0.000 ± 0.000
FTS5 lexical channel alone	0.142 ± 0.024	0.140 ± 0.020	0.164 ± 0.052	0.480 ± 0.005
Vectorize dense channel alone	0.330 ± 0.049	0.159 ± 0.022	0.713 ± 0.022	0.486 ± 0.000
Dense + cross-encoder rerank	0.282 ± 0.084	0.212 ± 0.021	0.721 ± 0.039	0.483 ± 0.005
RRF fusion, equal weights (<i>as deployed</i>)	0.287 ± 0.029	0.220 ± 0.028	0.538 ± 0.062	0.476 ± 0.008
RRF equal + rerank (<i>deployed pipeline</i>)	0.273 ± 0.058	0.232 ± 0.041	0.523 ± 0.053	0.480 ± 0.005
RRF fusion, lexical weight 0.3	0.325 ± 0.059	0.205 ± 0.026	0.689 ± 0.046	0.480 ± 0.005
RRF weighted + rerank	0.285 ± 0.077	0.223 ± 0.021	0.677 ± 0.040	0.483 ± 0.005

Table 3: nDCG@10 by query class, mean $\pm$ population standard deviation over 3 seeds. The classes separate sharply, and the separation is the point of the table. Paraphrase queries share almost no content words with the memory that answers them, so a lexical channel is close to blind there (0.164) while a dense channel is not (0.713). Temporal queries are the one class the lexical channel still matches on (0.480 against 0.486), because the token that distinguishes one link of a supersession chain from the next is usually a literal identifier. Query counts per seed are lexical 37, pinpoint 39, semantic 37, temporal 37.

the dense channel alone. Fusion is therefore sound on this workload and the deployed weights are wrong. That is a cheaper defect to fix than a redesign, and it is the main practical recommendation of this paper.

The judgement stage is a trade, not an improvement.

We report this against our own prior expectation. Applied over the dense shortlist, the reranker raises recall@10 from 0.598 to 0.620 and nDCG@10 marginally from 0.418 to 0.422, but it *lowers* precision@1 from 0.282 to 0.276. Over the re-weighted shortlist it raises recall@10 again (0.624 to 0.629) while *lowering* nDCG@10 from 0.422 to 0.415. The consistent effect is on the *pinpoint* class, where it is the largest single improvement anywhere in the table: 0.159 to 0.212 over the dense channel and 0.205 to 0.223 over the re-weighted fusion. A cross-encoder is good at deciding whether one specific passage answers one specific narrow question, which is what *pinpoint* queries are, and mediocre at ranking broad topical neighbourhoods, which is what it is being asked to do everywhere else.

So the stage is worth having for narrow factual lookups and not obviously worth its latency for general recall, and the honest summary is that it moves recall@10 up and rank-precision sideways or down. The architectural argu-

ment for the stage is unchanged and does not depend on this column: it is the only component whose behaviour improves with more decision data. We flag that the deployed pipeline, equal-weight fusion plus rerank, is the weakest of the eight configurations at 0.375, and that both changes above are configuration, not architecture.

Temporal queries are not where fusion earns its keep.

Every configuration except the recency control scores between 0.476 and 0.486 on *temporal*, including lexical retrieval alone at 0.480. The distinguishing token between one generation of an evolving fact and the next is usually a literal identifier, which is precisely what an exact-match channel is for. The temporal capability of the system comes from the supersede mechanism keeping stale rows retrievable at all, not from the ranking policy choosing well among them. That is worth stating plainly, because the temporal class is the one place a two-channel design would be expected to justify itself, and it does not.

Cost. The rerank stage issues one scoring call per short-listed candidate. In the harness this is a local cross-encoder on CPU at 0.56–0.85 s per query, measured across three seeds; it scores the union of the three shortlists compared

here (about 27 rows) so that fusion weight and rerank can be varied independently. Production issues a single batched call over the 20-row shortlist. Either way this is the dominant read-path cost, and it is the reason the shortlist is bounded by $\max(2k, 12)$ rather than by the store.

9 Comparison

Table 4 positions the system against the alternatives from section 2.2. The axes are chosen because they decide whether a service is operable, not because they flatter it.

The row we would defend hardest is *contradiction handling*. Accumulation is the simpler and safer default, and it is the right default when a wrong overwrite is unrecoverable. It is the wrong default when memories are configuration facts, because “the service uses Kafka” and “the service uses NATS” cannot both be current, and returning both with equal confidence is a failure mode users experience as the system being unreliable rather than wrong. The supersede mechanism addresses this while keeping the prior row retrievable, so the cost of a wrong supersession is bounded.

10 Operational Design

Three operational concerns are worth recording because they are load-bearing for correctness rather than merely for billing.

Quota authority. Quotas are decided in one place. A per-account override, when present, wins; otherwise the plan decides. An earlier revision advertised one number and enforced another — the free tier was documented at 1000 requests while the enforcement path returned 5600 — which is a bug that no test catches, because both numbers are plausible. The lesson generalises: display and enforcement must read the same value.

Bounded input. The content field is capped at 64,000 characters. This is a cost control, not a style preference: embedding and reasoning both bill per token while pay-as-you-go is billed per request, so an unbounded field lets a single call cost far more than the price charged for it. The bound puts the worst case at roughly a third of a cent of model time.

Token lifecycle. Refresh tokens carry their own expiry column, separate from the access token, because conflating the two produces either silent logouts or refresh tokens that never expire. Rotation is idempotent so that a retrieved refresh cannot mint a second valid token.

11 Limitations and Threats to Validity

We are explicit about what this paper does not establish.

The evaluation is a simulation. The workload is synthetic and templated. Its vocabulary is narrower than a real engineer’s, and its distractors are less adversarial than documents that were written to be confusing. The absolute numbers should not be compared to any published benchmark. The *relative* ordering of the systems is the durable result, and even that is conditioned on the query taxonomy: the *semantic* class is built by substituting synonyms, which favours dense retrieval by construction.

The reranker is not JEV. The judgement stage is represented by a cross-encoder — the query-document-scoring architecture of Devlin et al. [2019] and the general-purpose text embedding family of Wang et al. [2024] stand in for a model we could not run offline. It shares the architecture — score a bounded shortlist — but not the training signal, not the calibration, and not the cost. Numbers for that row estimate the value of the slot in the architecture, not the quality of the model that fills it.

Write-side judgements are generated, not inferred. Because type and salience are rule-generated in the harness, the salience term in eq. (3) is exercised but not validated. The gate constant of 3.0 was calibrated against a live score distribution that we cannot reproduce offline.

Single-tenant scale. The evaluation runs hundreds of rows. The design claims scale through serverless concurrency and the bounded shortlist, not through a measured throughput result, and we did not measure one.

Redaction completeness. Section 7 argues for redaction at enqueue and states the residual risk. We have no measurement of its recall against secrets in unfamiliar formats.

12 Conclusion

Agent memory is usually presented as a retrieval problem with a vector index at the centre. The harder problem is deciding what to keep and what to believe when a store accumulates, and the two channels that make retrieval robust — dense and lexical — fail in complementary ways that reward fusion rather than selection. Above that, a bounded judgement stage over the fused shortlist is the component that can improve with data, and logging every decision it makes turns a runtime detail into a research artifact. DGUI-HyperMem implements this on a serverless runtime with an open corpus and an MCP-native surface; the retrieval evaluation, while a simulation rather than a DeepSWE result, supports the ordering of its components and shows exactly where each one breaks.

Reproducibility

The reasoning corpus and its schema are public (ctaxnagomi/DGUI_HYPERMEM-JEV)[Hosen and CTECX Development & Research, 2026]; each row

Table 4: Agent memory systems. “Self-host” means the operator can run it on infrastructure they already control. “Reasoning at read” indicates a model call that re-orders a retrieved shortlist. Public corpus indicates that the system’s own decisions are exported in inspectable form.

System	Retrieval	Self-host	MCP	Reasoning at read	Co
MCP knowledge-graph server [Model Context Protocol, 2025]	graph, local	yes	yes	no	
mem0 [Chhikara et al., 2025]	dense + BM25 + entity	yes	yes	no	accur
Zep / Graphiti [Zep Software, 2025]	temporal graph	OSS engine	via plugin	no	
Letta [Letta AI, 2025]	agent memory blocks	yes	n/a	agent-driven	
Cognee [Cognee, 2025]	graph + vector	yes	partial	partial	
DGUI-HyperMem	dense + BM25, RRF	Workers	native	yes, bounded shortlist	au

Table 5: Plans. Quota is monthly requests; pay-as-you-go and credit packs cover bursts beyond the tier.

Plan	Requests/mo	Price
Free	2,000	\$0
Median	8,500	\$2.99
Pro	15,000	\$11.99
Enterprise	25,000	\$29.99

records the instruction, input, and answer of a real decision, redacted at enqueue. The retrieval harness for section 8 ships with this document in eval/: a seeded generator (evalsim.py), the runner that scores the eight configurations of table 2 (evalsim_run.py), the generator for this paper’s tables (mkresults.py), and a checker (verify_prose.py) that re-derives every number quoted in section 8 from the raw results and fails loudly on a mismatch. It is deterministic given its seed and embedding model, and reconstructs the constants of eq. (2) and eq. (3) directly from the shipped implementation. The service itself is MIT-licensed and self-hostable.

References

- Anthropic. Model context protocol: Specification. <https://modelcontextprotocol.io>, 2025.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms concordet and individual rank learning methods. *Information Retrieval Journal*, 16(3):213–240, 2009. doi: 10.1007/s10778-008-9021-7.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. arXiv:2504.19413.
- Zep Software. Zep cloud: Temporal knowledge graph memory for agents. <https://github.com/getzep/zep>, 2025. Graphiti open-source engine at <https://github.com/getzep/graphiti>.
- Letta AI. Letta: Stateful agents with memory. <https://github.com/letta-ai/letta>, 2025. Formerly MemGPT.
- Model Context Protocol. Knowledge graph memory server (reference implementation). <https://github.com/modelcontextprotocol/servers>, 2025. @modelcontextprotocol/server-memory.
- Cloudflare. D1: Serverless SQL on Cloudflare. <https://developers.cloudflare.com/d1>, 2025a.
- Cloudflare. Vectorize: Vector database on Cloudflare. <https://developers.cloudflare.com/vectorize>, 2025b.
- Cloudflare. Cloudflare Workers developer platform. <https://developers.cloudflare.com/workers>, 2025c.
- DeepSWE. DeepSWE: A software-engineering agent benchmark. <https://github.com/deep-swe>, 2026.
- NanJun Ding, Xiaoyu Xu, Yujia Wang, Xian Hu, Hongxia Deng, Bei Zhao, and Haohan Wang. Robust Dense passage retrieval via low-intensity Query expansion. In *Proceedings of the 2022 Conference of the Association for the Advancement of Artificial Intelligence (AAAI)*, 2022.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- Cognee. Open-source AI memory platform for agents. <https://github.com/topoteretes/cognee>, 2025.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186, 2019.
- Liang Wang, Nan Yang, Xiaolong Huang, Binning Sun, Furu Wei, Baoxiang Zhou, and Hexiang Lin. Improving text embeddings with large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. General text embedding (GTE) recipe.
- Wan Mohd Azizi bin Wan Hosen and CTECX Development & Research. DGUI-HyperMem: Hybrid long-term

memory MCP service. Technical report, CTECX Development & Research, 2026. Service documentation: `dgui-hypermem.deckergui.my`.